LLM Serving and Inference Optimization

Key Takeaways

LLMs are memory-bandwidth bound, not compute-bound

For large models, generating each token requires loading all weights from GPU HBM memory. Buying a faster GPU (more FLOPS) helps less than reducing model size in memory or improving memory access patterns. Quantization and fused kernels attack the right bottleneck.

The KV cache is the biggest memory hog in production

Storing Keys and Values for all previous tokens can require 100 GB per active request for large models. PagedAttention (vLLM) eliminates memory fragmentation by treating KV cache like OS virtual memory pages, enabling 24× higher throughput on the same hardware.

Quantization is often "almost free"

Moving from fp16 to INT8 or INT4 using GPTQ or AWQ cuts memory by 2–4× with only 3–7% perplexity increase. A 70B model in INT4 fits on a single 40 GB A100 that could not hold the fp16 version at all. Always validate on your specific task before shipping.

Continuous batching is the industry standard for GPU utilization

Static batching leaves GPUs 70–80% idle because all sequences in a batch must wait for the longest one. Continuous batching refills slots as sequences finish, pushing GPU utilization to 70–85% and multiplying throughput by 3–4× at zero quality cost.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. A 7B LLM at fp16 occupies 14 GB of VRAM. You switch to INT4 quantization using AWQ. How much VRAM do the weights now occupy, and what is the rough quality trade-off you accept?

Q2. Explain in two sentences why KV cache causes memory fragmentation in a naive serving system, and how PagedAttention solves it.

Q3. You are building a product that calls an LLM 5 million times per day with an average of 800 output tokens per call. At $0.060 per 1,000 output tokens, what is the monthly API cost? At what point should you seriously evaluate self-hosting?

Q4. Speculative decoding uses a small draft model and a large verifier model. Why can the verifier process all K draft tokens in a single forward pass, when normal generation requires K sequential passes?

Coding / Design Exercises

Work through these independently or in pairs. Show your working — the numbers matter.

Exercise 1: Memory Footprint Estimation for LLaMA-2-70B

Part A — Weights only. Calculate the RAM required to load LLaMA-2-70B weights in (i) fp16 and (ii) INT4. Show the formula: params × bytes/param.

Part B — KV cache. LLaMA-2-70B has 80 transformer layers, 64 attention heads, and a head dimension of 128. For a 4096-token context at fp16, calculate the KV cache memory per request using the formula: $2 \times L \times H \times d \times S \times bytes$.

Part C — Concurrent requests. With an 8×A100 cluster (8×80 GB = 640 GB total VRAM), how many concurrent requests can you serve in fp16 given the weights + KV cache per request from Parts A and B? How does this change with INT4 weights?

Exercise 2: Design a Serving Stack for 10M LLM Calls per Day

You are the ML engineer for a product making 10 million LLM calls per day with a p99 latency requirement of 200 ms. Average request: 200 input tokens, 500 output tokens.

Design a serving stack that meets this requirement. Your answer should cover: (1) model choice and quantization level; (2) serving framework (vLLM, TGI, or other — justify); (3) batching strategy; (4) number of GPUs needed (show throughput math); (5) one monitoring metric you would alert on.

Hint: vLLM on a single A100 at INT4 can serve ∼2,000 tokens/sec with continuous batching. Work out how many A100s the throughput requires before worrying about latency.

Challenge Problem

Cost Comparison: GPT-4o API vs. Self-Hosted LLaMA-3-70B at 100M tokens/day

You need to serve 100 million output tokens per day. Compare two options:

Option A — GPT-4o API: $60 per 1M output tokens. No infrastructure cost. Calculate the monthly bill.

Option B — Self-hosted LLaMA-3-70B on 8×A100:

- Cloud price: $25/hour for an 8×A100 instance
- With INT4 + continuous batching: ∼4,000 tokens/sec throughput
- 100M tokens/day ÷ 4,000 tok/sec = how many GPU-hours?
- Add 20% overhead for reliability (spare capacity, restarts)
- Engineering cost: assume 0.5 FTE at $200K/year fully loaded

Calculate the monthly GPU cost + engineering cost for Option B. What is the break-even volume between the two? At what volume does each option win? State any assumptions you make.

Further Reading

- **Kwon et al. (2023) — Efficient Memory Management for LLM Serving with PagedAttention** — SOSP 2023, available at vllm.ai
  The original PagedAttention paper. Read the introduction and Section 3. The main idea (non-contiguous KV cache blocks) is accessible without deep systems knowledge.
- **Frantar et al. (2022) — GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers** — arXiv 2210.17323
  The GPTQ paper. Focus on the intuition in Section 2 (OBQ framework) — the key insight is error compensation across weights in the same layer.
- **Lin et al. (2023) — AWQ: Activation-aware Weight Quantization for LLM Compression** — arXiv 2306.00978
  Short paper. Section 3.1 ("salient weight" observation) is the most important part. Explains why protecting 1% of weights by activation magnitude recovers most of the quality.
- **Chip Huyen — Designing Machine Learning Systems, Ch. 7** — O'Reilly, 2022 (course primary text)
  The model optimization and serving chapter. Covers quantization, distillation, and hardware trade-offs in the context of production systems.

Online Resources

- **vLLM Documentation** — docs.vllm.ai — production LLM serving, PagedAttention, continuous batching
- **llama.cpp GitHub** — github.com/ggerganov/llama.cpp — GGUF format, CPU inference, quantization levels
- **Hugging Face TGI** — huggingface.co/docs/text-generation-inference — alternative to vLLM
- **Tim Dettmers blog** — timdettmers.com — deep dives on INT8/INT4 quantization and GPU memory
- **Lilian Weng blog** — lilianweng.github.io — "LLM Inference Optimization" post (2023)
- **Andrej Karpathy — "Let's build the GPT Tokenizer"** — YouTube — helps build intuition for token counts

Answers

A1. INT4 uses 0.5 bytes/param. $7B \times 0.5 = \mathbf{3.5\,GB}$ (down from 14 GB at fp16, a $4\times$ reduction). The quality trade-off is a small increase in perplexity (roughly 3–7% for a well-tuned AWQ 4-bit quantization); for most summarization and instruction-following tasks this is imperceptible, but you should always evaluate on your specific task and test set before shipping.

A2. In a naive system, each request pre-allocates a contiguous buffer for its maximum possible context length (e.g. 100 GB for GPT-3 at 2048 tokens). Most requests use only a fraction of this, leaving the rest wasted and unavailable to other requests — this is internal fragmentation. PagedAttention solves it by dividing the KV cache into fixed-size blocks and allocating them on demand as generation proceeds, so memory is only consumed for tokens that have actually been generated.

A3. Output tokens per day: $5M \times 800 = 4 \times 10^9$. Monthly: $4 \times 10^9 \times 30 = 1.2 \times 10^{11}$ tokens. Cost: $1.2 \times 10^{11} \div 10^3 \times \$0.060 = \mathbf{\$7{,}200{,}000/month}$. At this volume (far above \$10K/month), self-hosting is almost certainly worth the engineering investment. Even if self-hosting costs \$1M/month in hardware and engineering, you save \$6.2M.

A4. During normal autoregressive generation the model must produce token t_k before it can process t_{k+1} (each token is an input to the next step). During speculative decoding verification, the draft tokens are already known — the large model uses teacher-forcing, computing $P(t_i|t_1 \ldots t_{i-1})$ for all positions simultaneously in a single batched forward pass, just as a language model does during training. There is no sequential dependency because the inputs are provided, not generated.

Course materials: priyanshsaxena.com/sst-computer-vision **Instructor:** Priyansh Saxena